

In [1]: `from google.colab import files`

`uploaded = files.upload()`

Choose files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving sales_data.csv to sales_data (1).csv

In [2]: `import pandas as pd`

`df = pd.read_csv('/content/sales_data.csv')`

`print("Dataset loaded successfully!")`

`print("Shape:", df.shape)`

Dataset loaded successfully!

Shape: (1000, 14)

In [5]: *#QUESTION 1 – Data Ingestion*

`import pandas as pd`

`import os`

`df = pd.read_csv("sales_data.csv")`

`print("First 5 Records:")`

`display(df.head())`

First 5 Records:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category
0	1052	2/3/2023	Bob	North	5053.97	18	Furniture
1	1093	4/21/2023	Bob	West	4384.02	17	Furniture
2	1015	9/21/2023	David	South	4631.23	30	Furniture
3	1072	8/24/2023	Bob	South	2167.94	39	Clothing
4	1061	3/24/2023	Charlie	East	3750.20	13	Electronics



In [6]: `print("Number of Rows:", df.shape[0])`

`print("Number of Columns:", df.shape[1])`

Number of Rows: 1000

Number of Columns: 14

In [7]: `print("Column Names:")`

`print(df.columns.tolist())`

Column Names:

['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount', 'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price', 'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep']

In [8]: `print("Data Types:")`

`print(df.dtypes)`

```
Data Types:
Product_ID          int64
Sale_Date           object
Sales_Rep           object
Region              object
Sales_Amount        float64
Quantity_Sold       int64
Product_Category    object
Unit_Cost           float64
Unit_Price          float64
Customer_Type       object
Discount            float64
Payment_Method      object
Sales_Channel       object
Region_and_Sales_Rep object
dtype: object
```

```
In [9]: os.makedirs("data/raw", exist_ok=True)

df.to_csv("data/raw/sales_data.csv", index=False)

print("Raw dataset saved successfully!")
```

Raw dataset saved successfully!

```
In [10... #QUESTION 2 – Data Profiling
!pip install ydata-profiling -q
```

```

_____ 0.0/400.8 kB ? eta -:-:-:-
_____ 266.2/400.8 kB 7.9 MB/s eta 0:00:01
_____ 400.8/400.8 kB 5.6 MB/s eta 0:00:00
_____ 296.7/296.7 kB 7.3 MB/s eta 0:00:00
_____ 3.1/3.1 MB 25.4 MB/s eta 0:00:00
_____ 682.5/682.5 kB 14.5 MB/s eta 0:00:00
_____ 105.4/105.4 kB 5.2 MB/s eta 0:00:00
_____ 72.0/72.0 kB 2.5 MB/s eta 0:00:00
```

```
In [ ]: from ydata_profiling import ProfileReport

profile = ProfileReport(
    df,
    title="Sales Dataset Profiling Report",
    explorative=True
)

profile.to_file("sales_profiling_report.html")

print("Profiling report generated successfully!")
```

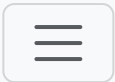
```
/tmp/ipykernel_5968/3861028190.py:1: DeprecationWarning:
`import ydata_profiling` is deprecated and will not receive more updates.
Please install fg-data-profiling via `pip install fg-data-profiling` and use `i
mport data_profiling` instead.

from ydata_profiling import ProfileReport
```

```
In [13... from IPython.display import display, HTML

display(HTML(filename="sales_profiling_report.html"))
```

Sales Dataset Profiling Report



Overview

Overview

Alerts7

Reproduction

Dataset statistics

Number of variables	14
Number of observations	1000
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	486.0 KiB
Average record size in memory	497.7 B

Variable types

Numeric	6
DateTime	1
Categorical	7

Variables

Select Columns

▼

Product_ID

Real number (ℝ)

Distinct

100

Distinct (%)

10.0%

Missing

0

Missing (%)

0.0%

Minimum

1001

Maximum

1100

Zeros

0

Zeros (%)

0.0%

Negative

0

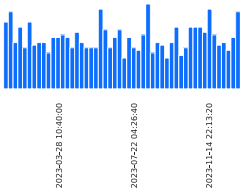
Infinite	0	Negative (%)	0.0%
Infinite (%)	0.0%	Memory size	7.9 KiB
Mean	1050.128		

More details

Sale_Date

Date

Distinct	340	Minimum	2023-01-01
Distinct (%)	34.0%	Maximum	2024-01-01
Missing	0	Invalid dates	0
Missing (%)	0.0%	Invalid dates (%)	0.0%
Memory size	7.9 KiB		



More details

Sales_Rep

Categorical

High correlation

Distinct	5	D...	222
Distinct (%)	0.5%	Eve	209
Missing	0	B...	208
Missing (%)	0.0%	A...	192
Memory size	52.4 KiB	C...	169

More details

Region

Categorical

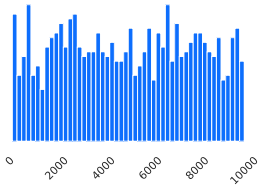
High correlation

Distinct	4	N...	267
Distinct (%)	0.4%	E...	263
Missing	0	W...	244
Missing (%)	0.0%	S...	226
Memory size	52.4 KiB		

More details

Sales_Amount

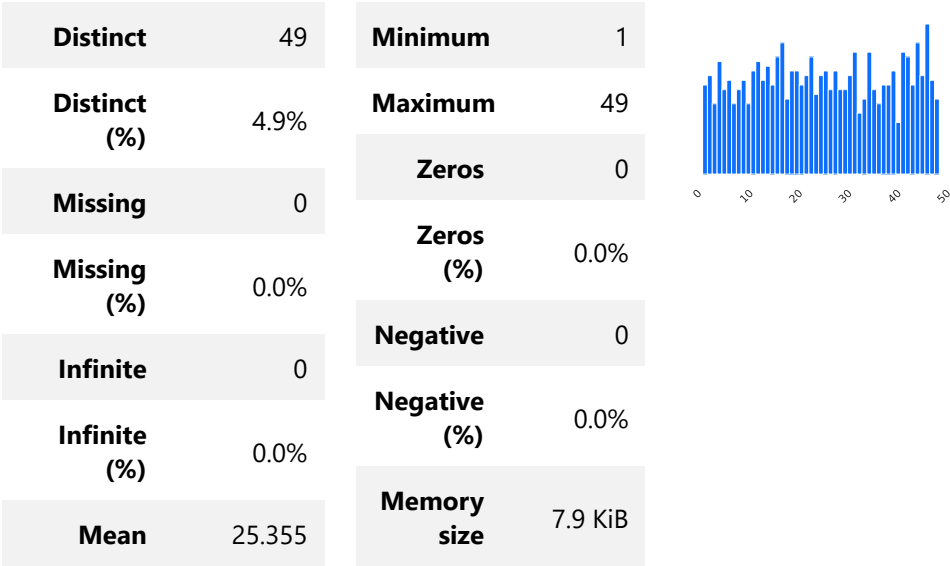
Real number (R)

Distinct	998	Minimum	100.12	
Distinct (%)	99.8%	Maximum	9989.04	
Missing	0	Zeros	0	
Missing (%)	0.0%	Zeros (%)	0.0%	
Infinite	0	Negative	0	
Infinite (%)	0.0%	Negative (%)	0.0%	
Mean	5019.2652	Memory size	7.9 KiB	

More details

Quantity_Sold

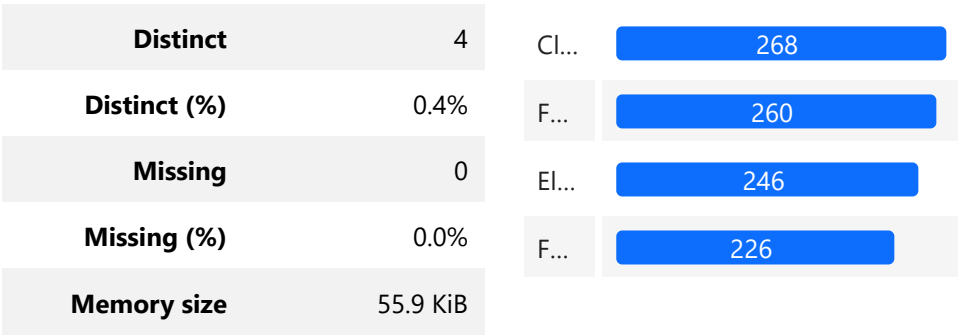
Real number (R)



More details

Product_Category

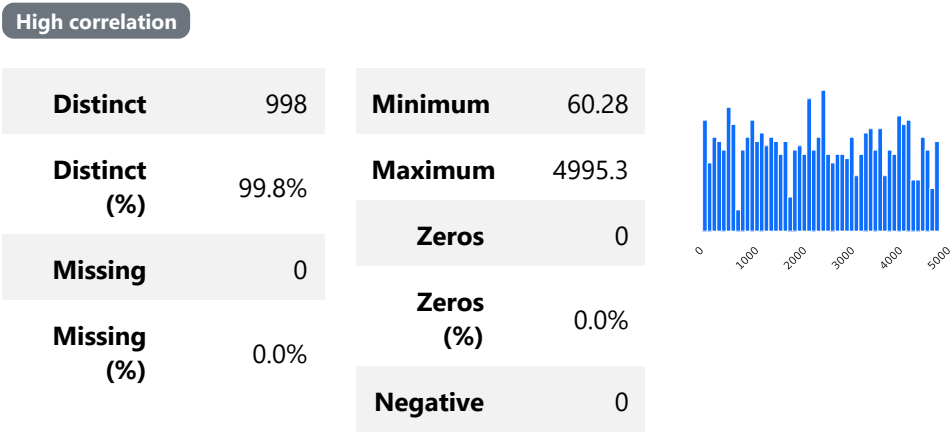
Categorical



More details

Unit_Cost

Real number ($\mathbb{R}$)



Infinite	0	Negative (%)	0.0%
Infinite (%)	0.0%	Memory size	7.9 KiB
Mean	2475.3045		

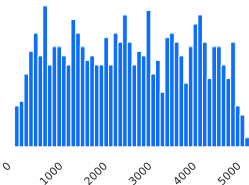
More details

Unit_Price

Real number (ℝ)

High correlation	Unique
Distinct	1000
Distinct (%)	100.0%
Missing	0
Missing (%)	0.0%
Infinite	0
Infinite (%)	0.0%
Mean	2728.4401

Minimum	167.12
Maximum	5442.15
Zeros	0
Zeros (%)	0.0%
Negative	0
Negative (%)	0.0%
Memory size	7.9 KiB



More details

Customer_Type

Categorical

Distinct	2
Distinct (%)	0.2%
Missing	0
Missing (%)	0.0%
Memory size	53.8 KiB

N...	504
R...	496

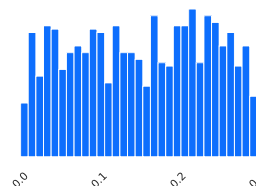
More details

Discount

Real number (ℝ)

Zeros

Distinct	31	Minimum	0
Distinct (%)	3.1%	Maximum	0.3
Missing	0	Zeros	16
Missing (%)	0.0%	Zeros (%)	1.6%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	0.15239	Memory size	7.9 KiB

[More details](#)

Payment_Method

Categorical

Distinct	3	C...	345
Distinct (%)	0.3%	B...	342
Missing	0	C...	313
Missing (%)	0.0%		
Memory size	57.3 KiB		

[More details](#)

Sales_Channel

Categorical

Distinct	2	R...	512
Distinct (%)	0.2%	O...	488

Missing	0
Missing (%)	0.0%
Memory size	53.8 KiB

More details

Region_and_Sales_Rep

Categorical

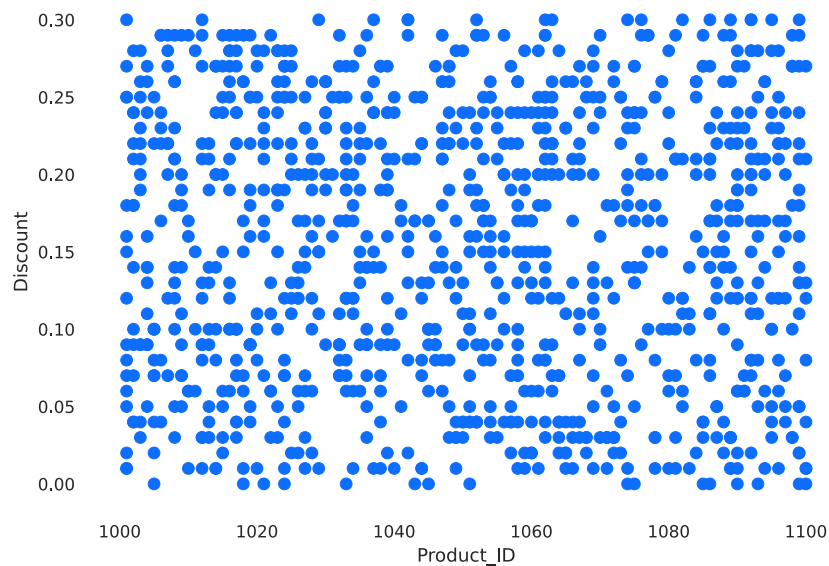
High correlation

Distinct	20	N...		64
Distinct (%)	2.0%	E...		60
Missing	0	E...		59
Missing (%)	0.0%	S...		56
Memory size	57.7 KiB	N...		56
		O...		705

More details

Interactions

- Product_ID• Sales_Amount• Quantity_Sold• Unit_Cost
 - Unit_Price• Discount
-
- Discount• Product_ID• Sales_Amount• Quantity_Sold
 - Unit_Cost• Unit_Price
-



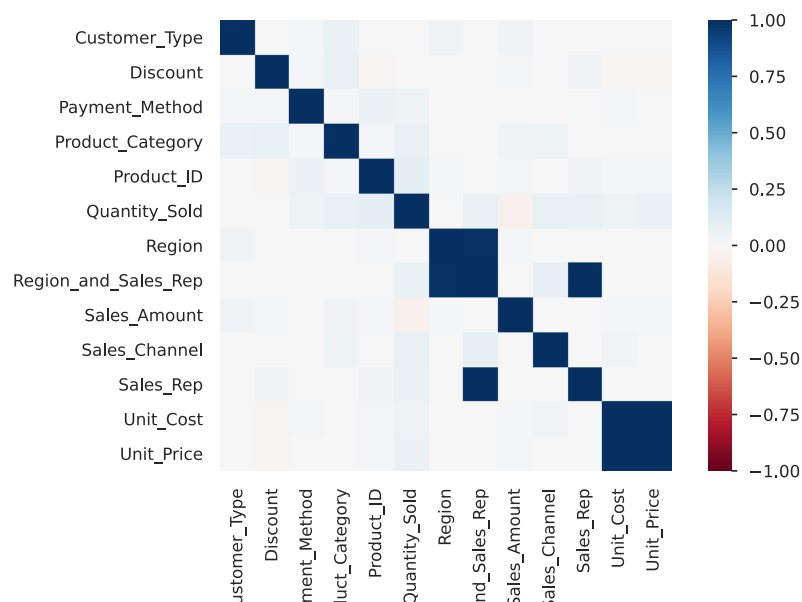
Correlations

•

Auto

•

Heatmap•

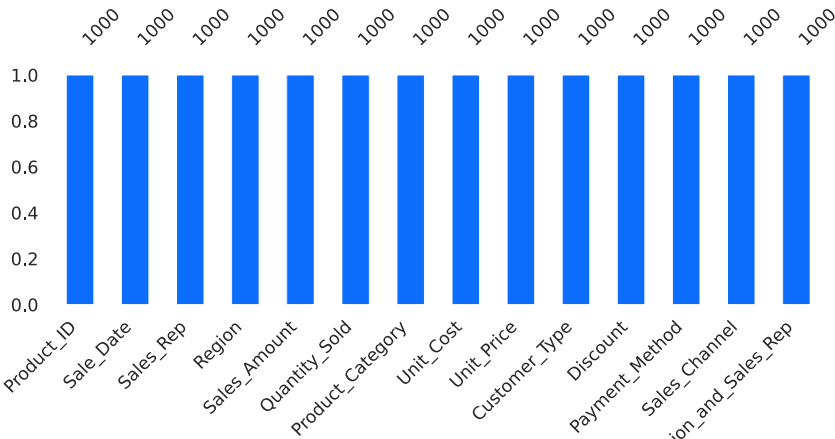
[Table](#)

Missing values

•

Count•

[Matrix](#)



A simple visualization of nullity by column.

Sample

First rows

Last rows

	Product_ID	Sale_Date	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep
0	1052	2/3/2023	North	5053.978	152	Furniture	152.72	267.22	Returning	0.09	Cash Online	North	
1	1093	4/21/2023	West	4384.027	384	Furniture	384.32	209.44	Returning	0.11	Cash Retail	West	
2	1015	9/21/2023	South	4631.230	261	Food	261.58	71.48	Returning	0.20	Bank Retail	South	
3	1072	8/24/2023	South	2167.939	430	Clothing	430.04	267.75	New	0.02	Credit Retail	South	
4	1061	3/24/2023	East	3750.203	613	Electronics	613.37	292.71	New	0.08	Credit Online	East	
5	1021	2/11/2023	West	3761.152	900	Food	900.79	106.51	New	0.21	Cash Online	West	
6	1083	4/11/2023	West	618.329	248	Furniture	248.26	24.09	Returning	0.14	Cash Online	West	
7	1087	1/6/2023	South	7698.945	379	Furniture	379.25	964.65	New	0.12	Bank Online	South	
8	1075	6/29/2023	South	4223.330	738	Furniture	738.00	1095.45	New	0.05	Bank Online	South	
9	1075	10/9/2023	West	8239.518	228	Clothing	228.25	82.34	New	0.13	Bank Online	West	

Report generated by [YData](#).

```
In [14... print("Missing Values:")
print(df.isnull().sum())
```

Missing Values:

Product_ID	0
Sale_Date	0
Sales_Rep	0
Region	0
Sales_Amount	0
Quantity_Sold	0
Product_Category	0
Unit_Cost	0
Unit_Price	0
Customer_Type	0
Discount	0
Payment_Method	0
Sales_Channel	0
Region_and_Sales_Rep	0
dtype:	int64

```
In [15... print("Number of Duplicate Records:", df.duplicated().sum())
```

Number of Duplicate Records: 0

```
In [16... for col in df.select_dtypes(include="object").columns:  
    print("\n", col)  
    print(df[col].unique())
```

Sale_Date

```
[ '2/3/2023' '4/21/2023' '9/21/2023' '8/24/2023' '3/24/2023' '2/11/2023'
'4/11/2023' '1/6/2023' '6/29/2023' '10/9/2023' '11/16/2023' '8/14/2023'
'11/11/2023' '12/31/2023' '8/17/2023' '10/16/2023' '5/30/2023'
'10/4/2023' '7/17/2023' '3/11/2023' '4/22/2023' '1/4/2023' '12/16/2023'
'11/27/2023' '11/14/2023' '4/5/2023' '6/1/2023' '11/7/2023' '5/17/2023'
'9/4/2023' '8/31/2023' '1/31/2023' '2/9/2023' '2/5/2023' '8/11/2023'
'11/18/2023' '8/8/2023' '3/16/2023' '12/1/2023' '5/14/2023' '4/28/2023'
'4/4/2023' '3/3/2023' '7/13/2023' '7/22/2023' '1/26/2023' '6/22/2023'
'10/27/2023' '5/13/2023' '11/22/2023' '11/20/2023' '10/2/2023'
'7/31/2023' '8/13/2023' '1/20/2023' '4/23/2023' '5/20/2023' '2/16/2023'
'1/1/2023' '3/31/2023' '5/22/2023' '2/7/2023' '10/20/2023' '5/19/2023'
'12/22/2023' '11/29/2023' '9/16/2023' '6/10/2023' '6/27/2023' '1/10/2023'
'9/18/2023' '5/18/2023' '6/21/2023' '1/13/2023' '6/17/2023' '5/10/2023'
'3/25/2023' '11/17/2023' '11/15/2023' '11/25/2023' '9/30/2023'
'9/22/2023' '11/1/2023' '3/10/2023' '11/30/2023' '3/17/2023' '2/4/2023'
'10/23/2023' '3/1/2023' '12/15/2023' '11/24/2023' '9/24/2023'
'12/27/2023' '9/27/2023' '11/12/2023' '10/13/2023' '10/18/2023'
'8/1/2023' '6/14/2023' '1/5/2023' '12/5/2023' '3/19/2023' '5/12/2023'
'3/30/2023' '8/25/2023' '10/22/2023' '4/14/2023' '1/14/2023' '10/15/2023'
'6/28/2023' '5/5/2023' '12/19/2023' '1/23/2023' '3/4/2023' '5/23/2023'
'6/2/2023' '5/25/2023' '11/5/2023' '2/8/2023' '3/8/2023' '1/15/2023'
'6/6/2023' '8/19/2023' '4/7/2023' '9/11/2023' '7/15/2023' '3/27/2023'
'5/3/2023' '11/26/2023' '10/21/2023' '6/8/2023' '10/30/2023' '8/16/2023'
'8/4/2023' '7/3/2023' '5/16/2023' '11/28/2023' '4/25/2023' '10/19/2023'
'10/26/2023' '10/10/2023' '5/28/2023' '1/11/2023' '12/26/2023'
'7/19/2023' '12/12/2023' '10/29/2023' '12/29/2023' '8/6/2023' '5/21/2023'
'7/29/2023' '6/20/2023' '5/7/2023' '4/16/2023' '4/2/2023' '10/14/2023'
'2/20/2023' '1/29/2023' '10/24/2023' '8/2/2023' '9/14/2023' '4/20/2023'
'7/1/2023' '11/9/2023' '2/24/2023' '5/29/2023' '7/21/2023' '4/27/2023'
'6/3/2023' '9/1/2023' '1/25/2023' '12/13/2023' '1/17/2023' '5/8/2023'
'12/30/2023' '9/8/2023' '12/10/2023' '6/26/2023' '10/12/2023' '6/5/2023'
'2/18/2023' '12/3/2023' '12/8/2023' '7/8/2023' '11/23/2023' '8/12/2023'
'8/3/2023' '7/30/2023' '2/1/2023' '6/7/2023' '5/24/2023' '1/19/2023'
'1/18/2023' '6/24/2023' '4/24/2023' '8/10/2023' '6/15/2023' '12/25/2023'
'5/11/2023' '9/23/2023' '1/1/2024' '6/11/2023' '3/28/2023' '1/8/2023'
'11/19/2023' '9/12/2023' '2/23/2023' '9/6/2023' '8/27/2023' '4/6/2023'
'3/29/2023' '4/30/2023' '1/7/2023' '9/20/2023' '5/31/2023' '3/26/2023'
'5/27/2023' '7/7/2023' '3/23/2023' '1/2/2023' '7/16/2023' '8/22/2023'
'7/14/2023' '4/29/2023' '9/13/2023' '2/22/2023' '12/21/2023' '10/7/2023'
'3/22/2023' '2/28/2023' '7/28/2023' '1/9/2023' '10/8/2023' '7/20/2023'
'9/3/2023' '10/5/2023' '8/23/2023' '4/19/2023' '4/8/2023' '5/26/2023'
'12/7/2023' '2/10/2023' '12/28/2023' '1/27/2023' '8/7/2023' '9/25/2023'
'7/9/2023' '5/2/2023' '2/15/2023' '3/5/2023' '11/6/2023' '7/12/2023'
'11/3/2023' '6/30/2023' '6/13/2023' '7/6/2023' '8/20/2023' '8/15/2023'
'11/8/2023' '9/7/2023' '3/14/2023' '5/9/2023' '8/5/2023' '3/2/2023'
'4/18/2023' '4/12/2023' '12/20/2023' '1/12/2023' '2/13/2023' '2/26/2023'
'3/21/2023' '1/16/2023' '7/18/2023' '6/25/2023' '4/13/2023' '10/28/2023'
'7/2/2023' '6/19/2023' '9/26/2023' '10/1/2023' '9/2/2023' '7/4/2023'
'9/29/2023' '12/6/2023' '3/6/2023' '1/3/2023' '2/21/2023' '9/17/2023'
'3/13/2023' '4/17/2023' '8/29/2023' '9/19/2023' '4/9/2023' '12/4/2023'
'6/9/2023' '3/9/2023' '12/23/2023' '4/15/2023' '5/4/2023' '2/6/2023'
'10/11/2023' '7/5/2023' '2/25/2023' '12/2/2023' '10/31/2023' '6/18/2023'
'2/14/2023' '12/11/2023' '7/10/2023' '6/4/2023' '11/10/2023' '9/15/2023'
'7/25/2023' '3/18/2023' '8/21/2023' '6/12/2023' '3/15/2023' '12/18/2023'
'8/30/2023' '7/11/2023' '1/22/2023' '11/2/2023' '7/23/2023' '2/2/2023'
'3/20/2023' '2/27/2023' '9/28/2023' '1/21/2023' '7/27/2023' '4/3/2023'
'6/23/2023' '2/19/2023' '1/24/2023' '7/24/2023' '4/26/2023' '11/13/2023'
'12/14/2023' '8/26/2023' '8/9/2023' '7/26/2023']
```

```

Sales_Rep
['Bob' 'David' 'Charlie' 'Eve' 'Alice']

Region
['North' 'West' 'South' 'East']

Product_Category
['Furniture' 'Food' 'Clothing' 'Electronics']

Customer_Type
['Returning' 'New']

Payment_Method
['Cash' 'Bank Transfer' 'Credit Card']

Sales_Channel
['Online' 'Retail']

Region_and_Sales_Rep
['North-Bob' 'West-Bob' 'South-David' 'South-Bob' 'East-Charlie'
 'West-Charlie' 'South-Eve' 'North-Eve' 'West-Eve' 'South-Alice'
 'South-Charlie' 'North-David' 'East-Bob' 'West-David' 'East-Eve'
 'North-Alice' 'East-David' 'West-Alice' 'East-Alice' 'North-Charlie']

```

In [17... *#QUESTION 3 – Data Validation using Pandera*
!pip install pandera -q

```

_____ 0.0/502.0 kB ? eta -:--:--
____ 112.6/502.0 kB 4.2 MB/s eta 0:00:01
_____ 501.8/502.0 kB 7.9 MB/s eta 0:00:01
_____ 502.0/502.0 kB 5.8 MB/s eta 0:00:00

```

In [18... **import** pandera **as** pa
from pandera **import** Column, Check

```

schema = pa.DataFrameSchema({
    "Sales_ID": Column(str, nullable=False),
    "Sales_Date": Column(str, nullable=False),
    "Product_ID": Column(str, nullable=False),
    "Customer_ID": Column(str, nullable=False),
    "Quantity_Sold": Column(float, Check.ge(0), nullable=False),
    "Unit_Cost": Column(float, Check.ge(0), nullable=False),
    "Unit_Price": Column(float, Check.ge(0), nullable=False),
    "Discount": Column(float, Check.in_range(0, 1), nullable=False),
    "Sales_Amount": Column(float, Check.ge(0), nullable=False)
})

print("Schema created successfully!")

```

Schema created successfully!

```
/usr/local/lib/python3.13/dist-packages/pandera/_pandas_deprecated.py:144: FutureWarning: Importing pandas-specific classes and functions from the top-level pandera module will be **removed in a future version of pandera**. If you're using pandera to validate pandas objects, we highly recommend updating your import:
```

```
'''
# old import
import pandera as pa

# new import
import pandera.pandas as pa
'''
```

If you're using pandera to validate objects from other compatible libraries like pyspark or polars, see the supported libraries section of the documentation for more information on how to import pandera:

https://pandera.readthedocs.io/en/stable/supported_libraries.html

To disable this warning, set the environment variable:

```
'''
export DISABLE_PANDERA_IMPORT_WARNING=True
'''

warnings.warn(_future_warning, FutureWarning)
```

```
In [19... import pandera as pa
from pandera import Column, Check

schema = pa.DataFrameSchema({
    "Sales_ID": Column(str, nullable=False),
    "Sales_Date": Column(str, nullable=False),
    "Product_ID": Column(str, nullable=False),
    "Customer_ID": Column(str, nullable=False),
    "Quantity_Sold": Column(float, Check.ge(0), nullable=False),
    "Unit_Cost": Column(float, Check.ge(0), nullable=False),
    "Unit_Price": Column(float, Check.ge(0), nullable=False),
    "Discount": Column(float, Check.in_range(0, 1), nullable=False),
    "Sales_Amount": Column(float, Check.ge(0), nullable=False)
})

print("Schema created successfully!")

Schema created successfully!
```

```
In [20... #QUESTION 4 – Data Engineering
clean_df = df.copy()

print("Original shape:", clean_df.shape)

clean_df = clean_df.drop_duplicates()

print("After removing duplicates:", clean_df.shape)

Original shape: (1000, 14)
After removing duplicates: (1000, 14)
```

```
In [21... print("Missing values before:")
print(clean_df.isnull().sum())
```

```
clean_df = clean_df.dropna()

print("\nMissing values after:")
print(clean_df.isnull().sum())
```

Missing values before:

Product_ID	0
Sale_Date	0
Sales_Rep	0
Region	0
Sales_Amount	0
Quantity_Sold	0
Product_Category	0
Unit_Cost	0
Unit_Price	0
Customer_Type	0
Discount	0
Payment_Method	0
Sales_Channel	0
Region_and_Sales_Rep	0

dtype: int64

Missing values after:

Product_ID	0
Sale_Date	0
Sales_Rep	0
Region	0
Sales_Amount	0
Quantity_Sold	0
Product_Category	0
Unit_Cost	0
Unit_Price	0
Customer_Type	0
Discount	0
Payment_Method	0
Sales_Channel	0
Region_and_Sales_Rep	0

dtype: int64

```
In [22... clean_df["Sales_Date"] = pd.to_datetime(
    clean_df["Sales_Date"],
    errors="coerce"
)

print(clean_df["Sales_Date"].dtype)
```

```

-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Sales_Date'

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
/tmp/ipykernel_5968/2323534675.py in <cell line: 0>()
      1 clean_df["Sales_Date"] = pd.to_datetime(
----> 2     clean_df["Sales_Date"],
      3     errors="coerce"
      4 )
      5

/usr/local/lib/python3.13/dist-packages/pandas/core/frame.py in __getitem__(self, key)
    4100         if self.columns.nlevels > 1:
    4101             return self._getitem_multilevel(key)
-> 4102         indexer = self.columns.get_loc(key)
    4103         if is_integer(indexer):
    4104             indexer = [indexer]

/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812         raise KeyError(key) from err
    3813     except TypeError:
    3814         # If we have a listlike key, _check_indexing_error will raise

KeyError: 'Sales_Date'

```

```

In [23... numeric_columns = [
    "Quantity_Sold",
    "Unit_Cost",
    "Unit_Price",
    "Discount",
    "Sales_Amount"
]

for col in numeric_columns:
    clean_df[col] = pd.to_numeric(
        clean_df[col],
        errors="coerce"
    )

```

```

    )

clean_df = clean_df.dropna()

print(clean_df.dtypes)

Product_ID          int64
Sale_Date           object
Sales_Rep           object
Region             object
Sales_Amount        float64
Quantity_Sold       int64
Product_Category    object
Unit_Cost           float64
Unit_Price          float64
Customer_Type       object
Discount            float64
Payment_Method      object
Sales_Channel       object
Region_and_Sales_Rep object
dtype: object

```

```

In [24... clean_df.to_csv("sales_processed.csv", index=False)

print("sales_processed.csv saved successfully!")

sales_processed.csv saved successfully!

```

```

In [31... #QUESTION 5 – Data Pipeline
import pandas as pd
import os

def ingest_data(file_path):
    print("1. Ingesting data...")
    return pd.read_csv(file_path)

def profile_data(df):
    print("2. Profiling data...")
    print("Shape:", df.shape)
    print("Missing values:")
    print(df.isnull().sum())
    print("Duplicates:", df.duplicated().sum())

def validate_data(df):
    print("3. Validating data...")

    numeric_columns = [
        "Quantity_Sold",
        "Unit_Cost",
        "Unit_Price",
        "Discount",
        "Sales_Amount"
    ]

    for col in numeric_columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

    print("Validation completed!")
    return df

```

```
def clean_data(df):
    print("4. Cleaning data...")

    df = df.drop_duplicates()
    df = df.dropna()

    return df

def transform_data(df):
    print("5. Transforming data...")

    # Corrected column name from 'Sales_Data' to 'Sale_Date'
    df["Sale_Date"] = pd.to_datetime(
        df["Sale_Date"],
        errors="coerce"
    )

    return df

def save_processed_data(df, output_path):
    print("6. Saving processed data...")
    df.to_csv(output_path, index=False)
    print("Saved:", output_path)

def main_pipeline():

    df = ingest_data("sales_data.csv")

    profile_data(df)

    df = validate_data(df)

    df = clean_data(df)

    df = transform_data(df)

    save_processed_data(
        df,
        "sales_processed.csv"
    )

    print("\nComplete pipeline executed successfully!")

main_pipeline()
```

```

1. Ingesting data...
2. Profiling data...
Shape: (1000, 14)
Missing values:
Product_ID          0
Sale_Date           0
Sales_Rep           0
Region             0
Sales_Amount        0
Quantity_Sold       0
Product_Category    0
Unit_Cost           0
Unit_Price          0
Customer_Type       0
Discount            0
Payment_Method      0
Sales_Channel       0
Region_and_Sales_Rep 0
dtype: int64
Duplicates: 0
3. Validating data...
Validation completed!
4. Cleaning data...
5. Transforming data...
6. Saving processed data...
Saved: sales_processed.csv

```

Complete pipeline executed successfully!

```

In [33... #QUESTION 6 – Feature Engineering
df = pd.read_csv("sales_processed.csv")

df["Gross_Sales_Value"] = (
    df["Quantity_Sold"] * df["Unit_Price"]
)

df["Total_Cost"] = (
    df["Quantity_Sold"] * df["Unit_Cost"]
)

df["Estimated_Margin"] = (
    df["Gross_Sales_Value"] - df["Total_Cost"]
)

df["Discount_Amount"] = (
    df["Gross_Sales_Value"] * df["Discount"]
)

df["Sales_Date"] = pd.to_datetime(df["Sale_Date"])

df["Year"] = df["Sales_Date"].dt.year
df["Month"] = df["Sales_Date"].dt.month
df["Day_of_Week"] = df["Sales_Date"].dt.dayofweek

display(df.head())

```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Categori
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture
2	1015	2023-09-21	David	South	4631.23	30	Furniture
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics

5 rows × 22 columns



```
In [35... df.to_csv("sales_feature_engineered.csv", index=False)
print("Feature engineered dataset saved!")
```

Feature engineered dataset saved!

```
In [36... #QUESTION 7 – DVC Data Versioning
!git init
```

```
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /content/.git/
```

```
In [37... !pip install dvc -q
```

```

470.1/470.1 kB 8.0 MB/s eta 0:00:00
79.3/79.3 kB 3.4 MB/s eta 0:00:00
451.2/451.2 kB 20.0 MB/s eta 0:00:00
41.8/41.8 kB 1.6 MB/s eta 0:00:00
45.7/45.7 kB 1.9 MB/s eta 0:00:00
163.1/163.1 kB 6.6 MB/s eta 0:00:00
214.2/214.2 kB 6.6 MB/s eta 0:00:00
57.3/57.3 kB 1.2 MB/s eta 0:00:00
118.1/118.1 kB 3.7 MB/s eta 0:00:00
74.2/74.2 kB 2.3 MB/s eta 0:00:00
1.5/1.5 MB 36.0 MB/s eta 0:00:00
50.9/50.9 kB 2.0 MB/s eta 0:00:00
381.2/381.2 kB 10.1 MB/s eta 0:00:00
87.1/87.1 kB 3.8 MB/s eta 0:00:00
45.5/45.5 kB 1.4 MB/s eta 0:00:00
222.8/222.8 kB 6.6 MB/s eta 0:00:00
69.4/69.4 kB 1.9 MB/s eta 0:00:00
62.8/62.8 kB 2.0 MB/s eta 0:00:00

```

In [38... `!dvc init`

Initialized DVC repository.

You can now commit the changes to git.

```

+-----+
|               |
|   DVC has enabled anonymous aggregate usage analytics.   |
|   Read the analytics documentation (and how to opt-out) here:   |
|   <https://dvc.org/doc/user-guide/analytics>   |
|               |
+-----+

```

What's next?

- Check out the documentation: <<https://dvc.org/doc>>
- Get help and share ideas: <<https://dvc.org/chat>>
- Star us on GitHub: <<https://github.com/treeverse/dvc>>

In [39... `!dvc add data/raw/sales_data.csv`

```

• Checking graph
Adding...: 0% 0/1 [00:00<?, ?file/s{'info': ''}]
!
      |0.00 [00:00,      ?file/s]

!
  0% |          |0/? [00:00<?,      ?files/s]

Adding data/raw/sales_data.csv to cache: 0% 0/1 [00:00<?, ?file/s]
Adding data/raw/sales_data.csv to cache: 0% 0/1 [00:00<?, ?file/s{'info': ''}]

Checking out /content/data/raw/sales_data.csv: 0% 0/1 [00:00<?, ?files/s]
      0% 0/1 [00:00<?, ?files/s{'info': ''}]
Adding...: 100% 1/1 [00:00<00:00, 6.12file/s{'info': ''}]

To track the changes with git, run:

    git add data/raw/.gitignore data/raw/sales_data.csv.dvc

To enable auto staging, run:

    dvc config core.autostage true

```

In [40... `!dvc add sales_processed.csv`

```

• Checking graph
Adding...: 0% 0/1 [00:00<?, ?file/s{'info': ''}]
!
      |0.00 [00:00,      ?file/s]

!
  0% |          |0/? [00:00<?,      ?files/s]

Adding sales_processed.csv to cache: 0% 0/1 [00:00<?, ?file/s]
Adding sales_processed.csv to cache: 0% 0/1 [00:00<?, ?file/s{'info': ''}]

Checking out /content/sales_processed.csv: 0% 0/1 [00:00<?, ?files/s]
      0% 0/1 [00:00<?, ?files/s{'info': ''}]
Adding...: 100% 1/1 [00:00<00:00, 19.98file/s{'info': ''}]

To track the changes with git, run:

    git add sales_processed.csv.dvc .gitignore

To enable auto staging, run:

    dvc config core.autostage true

```

In [41... `!git status`

On branch master

No commits yet

Changes to be committed:

(use "git rm --cached <file>..." to unstage)

```
new file:   .dvc/.gitignore
new file:   .dvc/config
new file:   .dvcignore
```

Untracked files:

(use "git add <file>..." to include in what will be committed)

```
.config/
.gitignore
data/
sales_data (1).csv
sales_data.csv
sales_feature_engineered.csv
sales_processed.csv.dvc
sales_profiling_report.html
sample_data/
```

```
In [42... !git add .
!git commit -m "Add sales dataset and DVC tracking"
```

Author identity unknown

*** Please tell me who you are.

Run

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

to set your account's default identity.

Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'root@ef4eb2d5a50d.(none)')

```
In [43... #QUESTION 8 – Prepare ML-Ready Data
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("sales_feature_engineered.csv")

X = df.drop(columns=["Sales_Amount"])

y = df["Sales_Amount"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (1000, 21)
y shape: (1000,)
```

```
In [44... X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
```

```
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (800, 21)
Testing data: (200, 21)

In [45... *#QUESTION 9 – Data Preprocessing*

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()
```

```
categorical_features = X.select_dtypes(
    include=["object"]
).columns.tolist()
```

```
print("Numerical Features:")
print(numeric_features)
```

```
print("\nCategorical Features:")
print(categorical_features)
```

Numerical Features:

```
['Product_ID', 'Quantity_Sold', 'Unit_Cost', 'Unit_Price', 'Discount', 'Gross_Sales_Value', 'Total_Cost', 'Estimated_Margin', 'Discount_Amount', 'Year', 'Month', 'Day_of_Week']
```

Categorical Features:

```
['Sale_Date', 'Sales_Rep', 'Region', 'Product_Category', 'Customer_Type', 'Payment_Method', 'Sales_Channel', 'Region_and_Sales_Rep', 'Sales_Date']
```

In [46...

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])

print("Preprocessing pipeline created successfully!")
```

Preprocessing pipeline created successfully!

In [47... *#QUESTION 10 – Model Training & Evaluation*

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", model)
])

model_pipeline.fit(X_train, y_train)

print("Model training completed!")

```

Model training completed!

In [48... `y_pred = model_pipeline.predict(X_test)`

```

print("Predictions generated successfully!")

```

Predictions generated successfully!

In [51... `mae = mean_absolute_error(y_test, y_pred)`

```

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)

print("Model Evaluation Results")
print("-----")
print("MAE :", mae)
print("RMSE :", rmse)
print("R² :", r2)

```

Model Evaluation Results

```

-----
MAE : 2861.2665425
RMSE : 3245.0924383072297
R² : -0.13951507497732973

```

In [50... `!pip install mlflow -q`

```

_____ 50.0/50.0 kB 1.6 MB/s eta 0:00:00
_____ 50.9/50.9 kB 1.5 MB/s eta 0:00:00
_____ 44.2/44.2 kB 1.6 MB/s eta 0:00:00
_____ 11.6/11.6 MB 38.1 MB/s eta 0:00:00
_____ 3.7/3.7 MB 44.4 MB/s eta 0:00:00
_____ 1.8/1.8 MB 34.8 MB/s eta 0:00:00
_____ 267.4/267.4 kB 9.5 MB/s eta 0:00:00
_____ 148.8/148.8 kB 7.5 MB/s eta 0:00:00
_____ 114.9/114.9 kB 4.6 MB/s eta 0:00:00
_____ 228.4/228.4 kB 12.0 MB/s eta 0:00:00
_____ 136.5/136.5 kB 6.0 MB/s eta 0:00:00
_____ 132.2/132.2 kB 4.9 MB/s eta 0:00:00
_____ 1.0/1.0 MB 26.8 MB/s eta 0:00:00
_____ 216.0/216.0 kB 7.5 MB/s eta 0:00:00
_____ 72.5/72.5 kB 3.4 MB/s eta 0:00:00

```

```
In [52... import mlflow
import mlflow.sklearn

mlflow.set_experiment("Sales_Amount_Prediction")

print("MLflow experiment created!")
```

2026/09/10 03:23:52 INFO mlflow.store.db.utils: Creating initial MLflow database tables...
2026/09/10 03:23:52 INFO mlflow.store.db.utils: Updating database tables
2026/09/10 03:23:58 INFO mlflow.tracking.fluent: Experiment with name 'Sales_Amount_Prediction' does not exist. Creating a new experiment.
MLflow experiment created!

```
In [53... !pip install -U mlflow scikit-learn -q
```

9.1/9.1 MB 38.6 MB/s eta 0:00:00

```
In [54... import mlflow
import mlflow.sklearn

print("MLflow Version:", mlflow.__version__)

MLflow Version: 3.16.0
```

```
In [55... print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
print("Model:", model_pipeline)
```

```

MAE: 2861.2665425
RMSE: 3245.0924383072297
R2: -0.13951507497732973
Model: Pipeline(steps=[('preprocessing',
                        ColumnTransformer(transformers=[('num',
                                                         Pipeline(steps=[('imputer',
                                                                              SimpleImputer(st
rategy='median')),
                                                         ('scaler',
                                                         StandardScaler
                                                         ())),
                                                         ('Product_ID',
                                                         'Quantity_Sold', 'Unit_Cost',
                                                         'Unit_Price', 'Discount',
                                                         'Gross_Sales_Value',
                                                         'Total_Cost',
                                                         'Estimated_Margin',
                                                         'Discount_Amount', 'Year',
                                                         'Month', 'Day_of_Week'])),
                                                         ('cat',
                                                         Pipeline(steps=[('imputer',
                                                                              SimpleImputer(st
rategy='most_frequent')),
                                                         ('encoder',
                                                         OneHotEncoder(ha
ndle_unknown='ignore'))])),
                        ('model', RandomForestRegressor(random_state=42))])

```

```

In [56... mlflow.set_experiment("Sales_Amount_Prediction")
print("Experiment: Sales_Amount_Prediction")

```

Experiment: Sales_Amount_Prediction

```

In [57... from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("model", model)
])

model_pipeline.fit(X_train, y_train)

print("Model training completed!")

```

Model training completed!

```
In [58... from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred = model_pipeline.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("RMSE:", rmse)
print("R2  :", r2)

MAE : 2861.2665425
RMSE: 3245.0924383072297
R2   : -0.13951507497732973
```

```
In [59... import mlflow
import mlflow.sklearn

# Set MLflow experiment
mlflow.set_experiment("Sales_Amount_Prediction")

with mlflow.start_run() as run:

    # Log Parameters
    mlflow.log_param("model", "RandomForestRegressor")
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("random_state", 42)

    # Log Metrics
    mlflow.log_metric("MAE", float(mae))
    mlflow.log_metric("RMSE", float(rmse))
    mlflow.log_metric("R2_Score", float(r2))

    # Log the complete Scikit-Learn pipeline
    mlflow.sklearn.log_model(
        sk_model=model_pipeline,
        name="sales_prediction_model",
        serialization_format="pickle"
    )

    print("MLflow run completed successfully!")
    print("Run ID:", run.info.run_id)
```

2026/09/10 03:24:29 WARNING mlflow.sklearn: Saving scikit-learn models in the pickle or cloudpickle format requires exercising caution because these formats rely on Python's object serialization mechanism, which can execute arbitrary code during deserialization. The recommended safe alternative is the 'skops' format. For more information, see: https://scikit-learn.org/stable/model_persistence.html

2026/09/10 03:24:39 WARNING mlflow.utils.environment: Encountered an unexpected error while inferring pip requirements (model URI: /tmp/tmpfsqnwkux/model/model.pkl, flavor: sklearn). Fall back to return ['scikit-learn==1.9.0']. Set logging level to DEBUG to see the full traceback.

MLflow run completed successfully!
Run ID: d1a9fc1beada44c4a14654a6be58f0f5

In [60... *#QUESTION 12 – Complete MLOps Pipeline*

```
print("✓ Data Ingestion")
print("✓ Data Profiling")
print("✓ Data Validation")
print("✓ Data Engineering")
print("✓ Feature Engineering")
print("✓ ML Data Preparation")
print("✓ Data Preprocessing")
print("✓ Model Training")
print("✓ Model Evaluation")
print("✓ MLflow Experiment Tracking")
print("✓ Model Artifact Saved")

print("\nSales Amount Prediction Pipeline Completed Successfully!")
```

- ✓ Data Ingestion
- ✓ Data Profiling
- ✓ Data Validation
- ✓ Data Engineering
- ✓ Feature Engineering
- ✓ ML Data Preparation
- ✓ Data Preprocessing
- ✓ Model Training
- ✓ Model Evaluation
- ✓ MLflow Experiment Tracking
- ✓ Model Artifact Saved

Sales Amount Prediction Pipeline Completed Successfully!